Estructuras de Datos II — Proyecto Unity Tower Defense

Guía de Integración y Sustentación de Grafos

Proyecto: Laboratorio-EST2 **Temas:** Grafos, Dijkstra, Kruskal (MST), Kahn (DAG), BFS/DFS

📘 GUÍA COMPLETA DE INTEGRACIÓN DE GRAFOS EN UNITY

Materia: Estructuras de Datos II

Proyecto: Laboratorio Tower Defense & Árboles (`Laboratorio-EST2`)

📑 ÍNDICE

1. Introducción y Objetivos Académicos
2. Fundamento Teórico: Representación de Grafos
3. Estructura Base en C#: `Grafo<T>`
4. Opción 1: Sistema de Rutas y Pathfinding Dinámico (Dijkstra / BFS / A*)
5. Opción 2: Red de Energía y Sinergia entre Torres (Árbol de Expansión Mínima - MST Kruskal)
6. Opción 3: Panel UI Interactivo y Simulador de Grafos
7. Opción 4: Árbol de Habilidades y Desbloqueos (DAG & Orden Topológico)
8. Estrategia de Combinación Recomendada
9. Guía de Sustentación Frente al Profesor (Preguntas Clave)
10. Roadmap de Implementación Paso a Paso

1. Introducción y Objetivos Académicos

En el proyecto actual ya cuentas con estructuras avanzadas: - **Árbol Binario de Búsqueda (ABB)** en `arbol.cs` para la gestión y ordenamiento de jugadores por ID. - **Interfaz Gráfica del Árbol** en `ControladorArbolUI.cs`. - **Mecánicas de Tower Defense** en `enemyMovement.cs` y `levelManager.cs`.

La incorporación de **Grafos $G = (V, E)$** complementa la materia demostrando: - Modelado de redes complejas donde los nodos tienen múltiples conexiones y ciclos (a diferencia de un árbol que es acíclico y con jerarquía estricta padre-hijo). - Algoritmos clásicos de recorrido (**BFS**, **DFS**). - Algoritmos de caminos mínimos ponderados (**Dijkstra**). - Algoritmos voraces / greedy para árboles de expansión mínima (**Kruskal** con conjuntos disjuntos **DSU**). - Ordenamiento y dependencias en grafos acíclicos dirigidos (**Topological Sort / Kahn**).

2. Fundamento Teórico: Representación de Grafos

Un grafo se compone de un conjunto de vértices V y aristas E.

Representación	Espacio en Memoria	Ventajas	Desventajas	Cuándo usarla
Lista de Adyacencia	$O(\|V\| + \|E\|)$	Óptimo para grafos dispersos (*sparse*), iteración rápida sobre vecinos.	Comprobar si existe arista (u, v) tarda $O(\text{grado}(u))$.	**Recomendada para mapas, rutas y redes en juegos**.
Matriz de Adyacencia	$O(\|V\|^2)$	Comprobación (u, v) en $O(1)$.	Desperdicia memoria si hay pocas aristas.	Ideal para grafos densos o panel educativo con pocos nodos (≤ 20).

---

3. Estructura Base en C#: `Grafo<T>`

Para mantener una arquitectura limpia y reutilizable en el proyecto, podemos definir una estructura de grafo genérica que servirá para todas las propuestas.

Componentes:

1. `NodoGrafo<T>` / `Vertice<T>`: Contiene el dato/ID y la lista de aristas salientes.
2. `Arista<T>`: Referencia al vértice destino y el peso (distancia, costo o tiempo).
3. `Grafo<T>`: Gestiona la colección de vértices y provee los algoritmos centrales.

```
using System;
using System.Collections.Generic;

[Serializable]
public class Arista<T>
{
    public NodoGrafo<T> Destino;
    public float Peso;

    public Arista(NodoGrafo<T> destino, float peso = 1f)
    {
        Destino = destino;
        Peso = peso;
    }
}

public class NodoGrafo<T>
{
    public T Valor;
    public string Id;
    public List<Arista<T>> Vecinos;

    public NodoGrafo(string id, T valor)
    {
        Id = id;
        Valor = valor;
        Vecinos = new List<Arista<T>>();
    }
}

public class Grafo<T>
{
    public Dictionary<string, NodoGrafo<T>> Nodos = new Dictionary<string, NodoGrafo<T>
    public bool EsDirigido;

    public Grafo(bool esDirigido = false)
    {
        EsDirigido = esDirigido;
    }

    public void AgregarNodo(string id, T valor)
    {
        if (!Nodos.ContainsKey(id))
            Nodos[id] = new NodoGrafo<T>(id, valor);
    }

    public void AgregarArista(string idOrigen, string idDestino, float peso = 1f)
    {
        if (Nodos.ContainsKey(idOrigen) && Nodos.ContainsKey(idDestino))
        {
            Nodos[idOrigen].Vecinos.Add(new Arista<T>(Nodos[idDestino], peso));
            if (!EsDirigido)
            {
                Nodos[idDestino].Vecinos.Add(new Arista<T>(Nodos[idOrigen], peso));
            }
        }
    }
}
```

4. Opción 1: Sistema de Rutas y Pathfinding Dinámico (Dijkstra / BFS)

> ⭐ ***Recomendación Principal*** *para integrar en el juego real.*

4.1. Diagnóstico del Código Actual

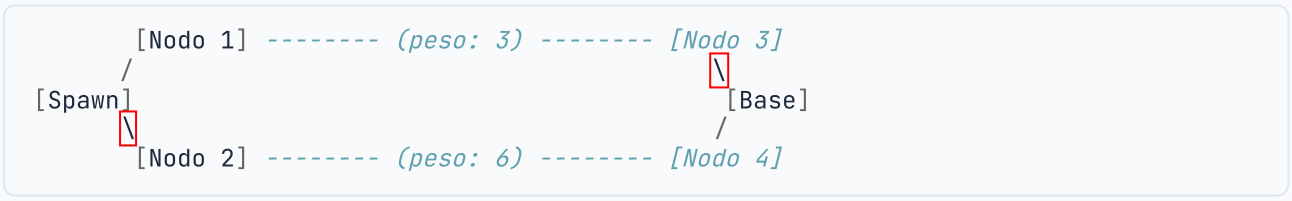
En `enemyMovement.cs`:

```
// El enemigo avanza estrictamente por un índice lineal 0, 1, 2...
pathIndex++;
target = levelManager.main.Path[pathIndex];
```

El arreglo `path` en `levelManager.cs` es estático y no permite: - Caminos alternativos o bifurcaciones. - Rutas más cortas según terreno o barro/lentitud (pesos). - Recalcular la ruta si una torre o trampa bloquea un nodo.

4.2. La Solución con Grafos

Convertimos los puntos del mapa en **Vértices de un Grafo** (`WaypointNode`). - **Nodos:** Puntos espaciales (`Vector3` / `Transform`) en el mapa. - **Aristas:** Conexiones transitables entre nodos. - **Peso de la Arista:** Distancia euclidiana (`Vector2.Distance(A, B)`) o costo de terreno. - **Algoritmo de Dijkstra:** Calcula la ruta de costo mínimo desde el Spawn hasta la Base.



Si el camino superior tiene peso total $2+3+2 = 7$ y el inferior $3+6+1 = 10$, el algoritmo de Dijkstra elegirá automáticamente la ruta superior. Si un obstáculo bloquea el camino superior, el grafo elimina temporalmente la arista y los enemigos recalculan hacia el camino inferior.

4.3. Algoritmo de Dijkstra Paso a Paso

1. Se inicializan las distancias desde el origen a todos los nodos con ∞, excepto el origen con 0.
2. Se usa una cola de prioridad (o lista ordenada) con pares `(nodo, distancia_acumulada)`.
3. Se extrae el nodo con menor distancia actual.
4. Para cada vecino: $$\text{nuevaDistancia} = \text{distanciaActual} + \text{pesoArista}$$ Si $\text{nuevaDistancia} < \text{distanciaRegistrada}$, se actualiza la distancia y se guarda el nodo predecesor para reconstruir el camino.
5. Al llegar al nodo destino, se reconstruye la ruta recorriendo los predecesores en reversa.

Complejidad: - Con lista simple: $O(V^2)$ - Con Min-Heap / PriorityQueue: $O((V + E) \log V)$

4.4. Componentes Visuales en Unity

- `OnDrawGizmos()`: Dibuja esferas en cada vértice y líneas de color entre vértices conectados.
- La ruta activa de cada enemigo se resalta con líneas amarillas o cian.

---

5. Opción 2: Red de Energía y Sinergia entre Torres (MST Kruskal)

5.1. Justificación Teórica

Demuestra el cálculo del **Árbol de Expansión Mínima (Minimum Spanning Tree - MST)**. - Dado un grafo conexo no dirigido y ponderado $G = (V, E)$, un MST es un subconjunto de aristas que conecta todos los vértices sin ciclos y con el peso total mínimo posible.

5.2. Mecánica en el Juego

Actualmente tienes la clase `torres` en `jugadores.cs`:

```
[System.Serializable]
public class torres {
    public int daño;
    // ...
}
```

Cuando el jugador coloca N torres en el mapa: 1. Cada torre es un vértice. 2. Todas las torres dentro de un radio forman aristas ponderadas por su distancia física. 3. El sistema ejecuta el **Algoritmo de Kruskal**: - Ordena todas las aristas de menor a mayor peso. - Usa la estructura **Union-Find (Disjoint Set Union - DSU)** para detectar si agregar una arista formaría un ciclo. - Si no forma ciclo, se une la conexión. 4. **Efecto de Juego:** Las torres interconectadas por la red de energía obtienen un bono de daño ($+15\%$) o atacan al mismo objetivo sincronizadamente. 5. **Visual:** Un componente `LineRenderer` dibuja un rayo de energía brillante entre las torres conectadas por el MST.

5.3. Estructura Union-Find (DSU) en C

```
public class DisjointSet
{
    private Dictionary<string, string> padre = new Dictionary<string, string>();

    public void MakeSet(string id) => padre[id] = id;

    public string Find(string id)
    {
        if (padre[id] != id)
            padre[id] = Find(padre[id]); // Compresión de caminos
        return padre[id];
    }

    public bool Union(string a, string b)
    {
        string raizA = Find(a);
        string raizB = Find(b);
        if (raizA == raizB) return false; // Ya están en el mismo conjunto (formaría ci
        padre[raizA] = raizB;
        return true;
    }
}
```

6. Opción 3: Panel UI Interactivo y Simulador de Grafos

6.1. Justificación Teórica

Siguiendo la misma filosofía de `ControladorArbolUI.cs`, este panel permite sustentar el tema de forma académica ante el docente mediante una interfaz gráfica manipulable en tiempo de ejecución.

6.2. Funcionalidades del Panel

- **Crear Vértice:** Input de texto para ingresar el nombre/ID del nodo (ej. "A", "B", "Base", "Torre1").
- **Crear Arista:** Seleccionar Origen, Destino y definir el Peso numérico.
- **Mostrar Matriz de Adyacencia:** Genera una tabla en pantalla donde las filas y columnas representan los nodos y las celdas muestran los pesos.
- **Botón Recorrido BFS (Anchura):** Muestra el orden de visita usando una Cola (`Queue<T>`).
- **Botón Recorrido DFS (Profundidad):** Muestra el orden de visita usando una Pila (`Stack<T>`) o recursión.
- **Botón Camino Mínimo (Dijkstra):** Seleccionar nodo inicio y fin, iluminando el camino más corto y el costo total.

6.3. Comparativa: BFS vs DFS

Característica	BFS (Breadth-First Search)	DFS (Depth-First Search)
Estructura Auxiliar	Cola (`Queue<T>` - FIFO)	Pila (`Stack<T>` - LIFO) o Recursión
Estrategia	Explora nivel por nivel (en anchura)	Explora tan profundo como sea posible antes de retroceder
Garantía Camino Mínimo	Sí (en grafos sin pesos o pesos uniformes)	No
Complejidad	$O(V + E)$	$O(V + E)$

---

7. Opción 4: Árbol de Habilidades y Desbloqueos (DAG & Orden Topológico)

7.1. Justificación Teórica

Un **Grafo Dirigido Acíclico (DAG)** modela sistemas de precedencia y requisitos. El **Ordenamiento Topológico** encuentra una secuencia lineal de vértices tal que para cada arista dirigida (u, v), el vértice u aparece antes que v.

7.2. Mecánica en el Juego

Para la clase `Jugador` en `jugadores.cs`: - Los jugadores ganan puntos de experiencia y pueden desbloquear mejoras: 1. *Balas Rápidas* $\rightarrow$ requiere *Nivel 1*. 2. *Torre de Hielo* $\rightarrow$ requiere *Balas Rápidas*. 3. *Torre Láser* $\rightarrow$ requiere *Torre de Hielo* y *Generador de Energía*. - Si se intenta crear un ciclo (ejemplo: A requiere B, y B requiere A), el sistema ejecuta el **Algoritmo de Detección de Ciclos** e impide el error lógico. - El **Algoritmo de Kahn** calcula el orden exacto en que las habilidades pueden ser desbloqueadas.

7.3. Algoritmo de Kahn (Topological Sort)

1. Calcular el grado de entrada (*in-degree*) de cada nodo.
2. Encolar todos los nodos con grado de entrada 0 (habilidades iniciales sin prerrequisitos).
3. Mientras la cola no esté vacía: - Extraer un nodo u y agregarlo al orden de desbloqueo. - Para cada vecino v, reducir su grado de entrada en 1. - Si el grado de entrada de v llega a 0, encolar v.
4. Si la cantidad de nodos procesados es menor que el total de nodos, **existe un ciclo**.

---

8. Estrategia de Combinación Recomendada

No es necesario elegir una sola opción de manera excluyente. Se pueden combinar de manera armónica:

```
[Núcleo Común: Grafo.cs, Vertice.cs, Arista.cs]
        │
        ├──► [Juego / Movimiento] : Opción 1 (Pathfinding Dijkstra para Enemigos)
        │
        ├──► [Torres / Mecánica]  : Opción 2 (MST Kruskal para Red de Sinergia)
        │
        └──► [Laboratorio / Demo] : Opción 3 (Panel UI para BFS, DFS y visualización)
```

1. **Opción 1 (Pathfinding con Dijkstra):** Transforma directamente la jugabilidad en `enemyMovement.cs` reemplazando la ruta fija por nodos conectados.
2. **Opción 2 (Red MST de Torres):** Agrega un elemento visual deslumbrante (rayos de energía entre torres) y demuestra el algoritmo de Kruskal + Union-Find.
3. **Opción 3 (Panel UI interactivo):** Asegura que si el profesor pide evaluar un grafo cualquiera con nodos arbitrarios, se pueda demostrar BFS, DFS y Dijkstra en vivo desde la pantalla.

---

9. Guía de Sustentación Frente al Profesor

Cuando defiendas tu proyecto, el docente te hará preguntas teóricas clave. Aquí tienes las respuestas fundamentadas:

P1: ¿Por qué usaron Lista de Adyacencia en lugar de Matriz de Adyacencia para el mapa?

> *"Porque el mapa del juego es un grafo disperso (sparse graph). En un mapa con 20 waypoints, cada nodo solo se conecta con 2 a 4 vecinos directos, nunca con los 20 a la vez. Una matriz de adyacencia de 20×20 gastaría memoria innecesaria con casillas en cero ($O(V^2)$), mientras que la lista de adyacencia consume solo $O(V + E)$ y permite recorrer los vecinos adyacentes en tiempo óptimo."*

P2: ¿Cuál es la diferencia entre BFS y Dijkstra?

> *"BFS encuentra el camino más corto únicamente si todas las aristas tienen el mismo peso (o no tienen peso), usando una simple cola FIFO en tiempo $O(V + E)$. Dijkstra es necesario cuando las aristas tienen pesos distintos (por ejemplo, distancias euclidianas diferentes o zonas con penalización de velocidad), garantizando el camino óptimo de menor costo acumulado mediante una estructura de prioridad."*

P3: ¿Por qué en Dijkstra los pesos deben ser no negativos?

> *"Porque Dijkstra asume vorazmente (algoritmo greedy) que una vez que un nodo es extraído con la distancia mínima acumulada, su costo no puede reducirse más. Si existieran aristas con pesos negativos, esta suposición se rompe y se requeriría el algoritmo de Bellman-Ford."*

P4: ¿Cómo garantiza Kruskal que no se formen ciclos en la red de torres?

> *"Kruskal utiliza la estructura de datos de Conjuntos Disjuntos (Disjoint Set Union o DSU) con operaciones `Find` y `Union`. Si los dos extremos de una arista ya pertenecen al mismo conjunto representativo (`Find(u) == Find(v)`), significa que ya existe un camino previo entre ellos; por lo tanto, agregar esa arista crearía un ciclo y es descartada."*

---

10. Roadmap de Implementación Paso a Paso

Cuando desees comenzar la implementación, seguiremos este orden estructurado:

- [ ] **Paso 1:** Crear el namespace y scripts base `Grafo.cs`, `NodoGrafo.cs` y `Arista.cs` en `Assets/codigos/grafos/`.
- [ ] **Paso 2:** Crear el componente `WaypointNode.cs` y `GrafoRutaManager.cs` con dibujo en Gizmos para el editor.
- [ ] **Paso 3:** Actualizar `enemyMovement.cs` para consumir la ruta calculada por Dijkstra en lugar del arreglo fijo `path[]`.
- [ ] **Paso 4:** Implementar el algoritmo de Kruskal en un administrador de torres (`RedTorresMST.cs`) con `LineRenderer`.
- [ ] **Paso 5:** Integrar la pestaña o panel UI para pruebas interactivas de BFS/DFS/Dijkstra.

Documento generado para el proyecto Laboratorio-EST2.